

CLOUD-158 — Load Balancer Health Checking That Reflects Real Service Capability

Canonical ID CLOUD-158	Dominant specialization SOFTWARE MODELLING & DEVOPS
Technical domain Software Modelling and DevOps	Strongest impact Industry Impact
Reference source: GitHub · HAProxy https://github.com/haproxy/haproxy	

READ THIS BEFORE CHOOSING THE PROJECT

Two-semester capstone project charter

CP1 + CP2 · complete engineering system

PROBLEM TO SOLVE

Load balancers remove unhealthy backends based on health checks, and a check that only confirms the process is listening will keep sending traffic to a backend whose database connection pool is exhausted. Consequently, the instance passes its check, fails every request, and stays in rotation. The consequence is unreliable service behaviour, difficult recovery or avoidable operational cost in Software, Cloud & Developer Infrastructure.

WHO BENEFITS AND WHAT OWNERSHIP IS ESTABLISHED

Primary beneficiaries are software developers and maintainers; the cited source establishes context or source custody but does not identify an operational problem owner. No sponsor, funder or supporting organisation for the KL student implementation is stated. The KL deployment stakeholder and operational decision owner must be confirmed before approval.

FINAL ACHIEVEMENT REQUIRED

Deliver health checking that reflects whether a backend can serve, without cascading failures through the check itself.

REFERENCE TO REPRODUCE IN CP1

Demonstrate a backend passing checks while failing requests. This is the O2 reference against which every Load Balancer Health Checking That Reflects Real Service Capability contribution must be compared.

DIFFERENTIATING CONTRIBUTION IN CP2

Design health checks reflecting real service capability. This is the O3 technical contribution that must be implemented, isolated and evaluated separately.

COMPLETE FINAL DEMONSTRATION

Avoid checks so deep they cascade failures across backends. The final demonstration must show the complete Load Balancer Health Checking That Reflects Real Service Capability workflow rather than an isolated component.

INDEPENDENT ACCEPTANCE

Validate check accuracy against actual request success. The result must be independently checked under representative, boundary and failure conditions.

WHY THIS IS CAPSTONE-WORTHY

This is a two-semester culminating project because the team must establish the O2 reference “Demonstrate a backend passing checks while failing requests”, design and evaluate the O3 contribution “Health checks reflecting real service capability”, integrate the O4 system “Avoid checks so deep they cascade failures across backends”, and defend the O5 result under independent, boundary and failure conditions.

Problem evidence and approval condition**OFFICIAL CONTEXTUAL EVIDENCE REQUIRING LOCAL PROBLEM VALIDATION**

Evidence locator: Load balancing and health checking · **Open cited source**

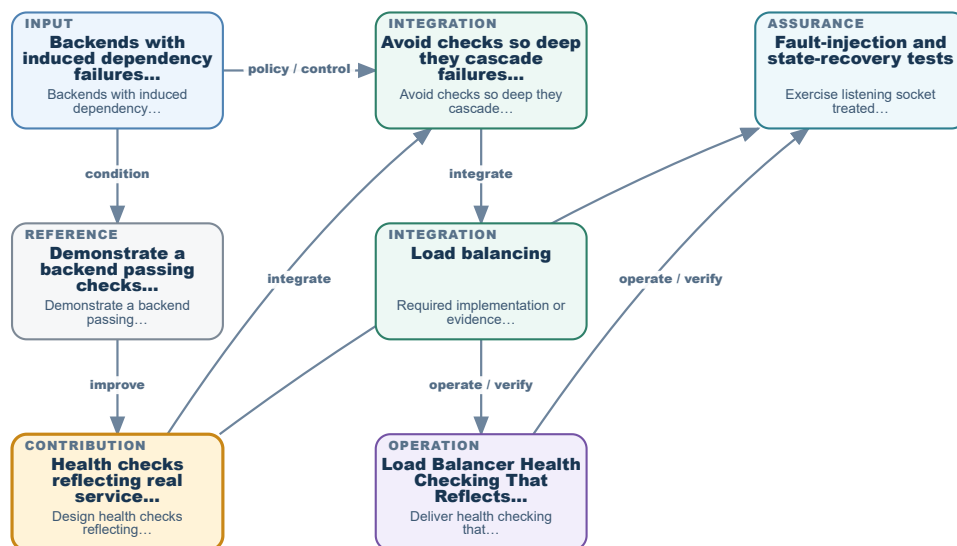
What the source establishes: The cited source establishes the engineering context, technology or application area; it does not by itself prove that the exact local problem remains unresolved.

What the team must verify: Before implementation proceeds, reproduce the stated limitation with versioned evidence or attach a signed problem-validation record from a named stakeholder. If neither is achieved, replace or rescope the project.

Pictorial project specification

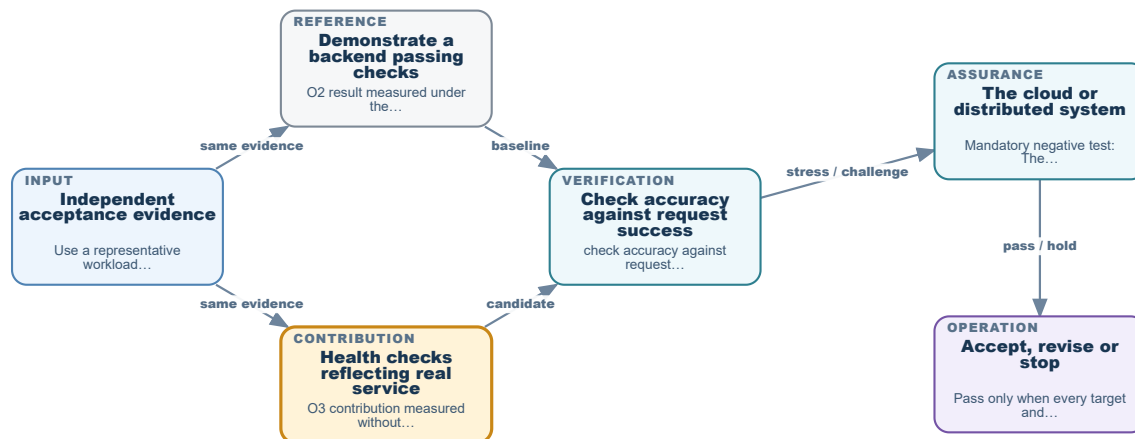
service topology

A. System / physical architecture — what the team must build and connect



Every arrow is an interface or evidence transfer that must be implemented and tested.

B. Validation architecture — how the team must prove the result



Every arrow is an interface or evidence transfer that must be implemented and tested.

Two-semester execution roadmap

Semester VII: problem + reference · Semester VIII: contribution + system + acceptance

SEMESTER VII / CAPSTONE PROJECT I

O1

Validate the problem and freeze the engineering contract

Balance traffic across backends with basic health checking. Verify the beneficiary, source claim, system boundary, evidence access, interfaces, hazards and acceptance rules before implementation proceeds.

CP1 Gate 1 — the problem, scope, evidence and acceptance plan are approved.

Mandatory outcome and evidence

Basic health-checked load balancing. Required evidence: source-to-claim map, stakeholder or reproduction evidence, requirements, architecture decision record, risk register and versioned manifests, workload traces, service telemetry, fault-injection and state-reconciliation evidence.

O2

Build the reproducible reference and vertical slice

Demonstrate a backend passing checks while failing requests. Compare at least two feasible reference approaches, select one through a documented trade-off and measure it under AC-1 and AC-4.

CP1 Gate 2 — a reproducible reference and an integrated vertical slice are demonstrated.

Mandatory outcome and evidence

Demonstrated check inadequacy. Deliver executable fixtures, an independent oracle, unit tests, resource profile, failure log and baseline values for check accuracy against request success; cascade failures avoided; unhealthy backend removal latency; successful recovery rate under fault injection (%).

SEMESTER VIII / CAPSTONE PROJECT II

O3

Engineer and prove the differentiating contribution

Design health checks reflecting real service capability. Compare it with O2 using the same evidence and resource envelope; include ablation, sensitivity, trade-off or controlled experiments appropriate to the project.

CP2 Gate 3 — the contribution is technically defensible and its trade-offs are measured.

Mandatory outcome and evidence

Capability-reflecting health checks. Report baseline-versus-candidate results, uncertainty, condition-wise failures and a defensible positive or negative conclusion. Retain the selected configuration, experimental controls, raw evidence and a trace from each claim to its acceptance result.

O4**Integrate the complete operable system**

Avoid checks so deep they cascade failures across backends. Connect the evidence path, reference and contribution, user or operator interface, configuration, security or safety controls, telemetry, degraded mode and recovery.

CP2 Gate 4 — the complete system operates normally, degrades safely and recovers.

Mandatory outcome and evidence

Cascade-safe check design. Every component and interface must have unit, integration and system evidence in a representative environment. Retain configuration, interface contracts, telemetry, degraded-mode behaviour and recovery proof.

O5**Complete independent acceptance and impact validation**

Validate check accuracy against actual request success. Execute AC-1 to AC-4 and all 5 negative tests; obtain independent domain, stakeholder or technical review.

CP2 Final Gate — every target and guardrail passes, or the project is explicitly held or revised.

Mandatory outcome and evidence

A validation report on check accuracy. Publish baseline, target and achieved values, limitations, residual risks, reproducibility package, operating guide, final report, demonstration video and individual contribution evidence.

Functional requirements

- FR-1 — Ingest or acquire “Backends with induced dependency failures and traffic”; validate version, provenance, schema or protocol, units and project-specific quality gates before use.
- FR-2 — Reproduce “Demonstrate a backend passing checks while failing requests” through a one-command, one-protocol or one-test-run reference path with an independent expected-result oracle.
- FR-3 — Implement “Health checks reflecting real service capability” as a separately testable subsystem whose configuration and evidence are versioned.
- FR-4 — Implement and verify “Avoid checks so deep they cascade failures across backends” and its input, output, error, configuration and telemetry contracts.
- FR-5 — Implement and verify “Load balancing” and its input, output, error, configuration and telemetry contracts.
- FR-6 — Deliver “Load Balancer Health Checking That Reflects Real Service” as the complete demonstrable result; no isolated notebook, model, function or dashboard counts as completion.
- FR-7 — Detect the condition described by NT-1, invoke the documented safe or degraded response and retain recovery evidence.

Non-functional and domain-assurance requirements

- NFR-1 — Performance: meet every accepted KPI target under AC-1 and AC-4, including tail values and resource cost where applicable.
- NFR-2 — Reliability and recovery: pass every mandatory negative test; a critical failure cannot be hidden by an average.
- NFR-3 — Domain assurance: The system shall define SLOs for availability, tail latency, recovery and data integrity under the approved load.
- NFR-4 — Evidence quality: Retries, timeouts, concurrency and partial failures shall not cause silent state loss or duplicate work.
- NFR-5 — Maintainability: use version control, automated tests, documented interfaces, configuration validation and a clean-environment build or protocol.
- NFR-6 — Operability: provide logs, measurements, health or quality indicators, operator guidance, degraded mode and a verified recovery procedure.

System boundary and permitted scope

INCLUDED

- Reproducible reference: Demonstrate a backend passing checks while failing requests. This is the O2 reference against which every Load Balancer Health Checking That Reflects Real Service Capability contribution must be compared
- Differentiating contribution: Design health checks reflecting real service capability. This is the O3 technical contribution that must be implemented, isolated and evaluated separately
- Integrated result: Avoid checks so deep they cascade failures across backends. The final demonstration must show the complete Load Balancer Health Checking That Reflects Real Service Capability workflow rather than an isolated component
- Independent acceptance: Validate check accuracy against actual request success. The result must be independently checked under representative, boundary and failure conditions

EXPLICITLY EXCLUDED

- out of scope.
- A notebook, isolated model, single-function prototype, dashboard-only demonstration or unvalidated library integration.
- Production certification, universal generalisation and unattended high-stakes operation outside the validated envelope.
- An isolated notebook, model, function, dashboard or library integration without the complete operating and validation workflow.

DEPENDENCY ASSUMPTION

Access to every required dataset, sample, device, tool, reviewer, licence and approval must be confirmed at the project-approval review; unavailable dependencies require an approved scope revision.

Team, prerequisites, tools and feasibility

RECOMMENDED TEAM

A team of 3–4 students must assign named responsibility for workload and failure evidence leadership, control-plane or platform leadership, service integration and observability leadership, reliability and recovery leadership in Load Balancer Health Checking That Reflects Real Service Capability; each member must own reviewable implementation and evidence.

PREREQUISITES

Load balancing, service architecture, failure analysis.

PRINCIPAL TOOLS

- load balancing
- health check design
- dependency analysis
- cascade prevention

FACILITY / COMPUTE

An isolated container or cluster testbed with workload generation, telemetry and fault injection; no production dependency is required. Principal access must include load balancing, health check design, dependency analysis, cascade prevention.

FALLBACK PATH

Use an isolated local cluster or deterministic simulator while preserving workload shape, failure semantics and telemetry. It may reduce scale but must retain O2, O3, O4 and O5.

Acceptance conditions

AC-1 Representative operation

Validate check accuracy against actual request success.

AC-2 Boundary and failure operation

Exercise listening socket treated as health; exhausted connection pools passing checks; deep checks cascading failures.

AC-3 Independent acceptance evidence

Use a representative workload and an independent fault campaign with versioned configuration and workload traces.

AC-4 Frozen resource envelope

Use the same approved versions, workload, hardware or facility limits and resource budget for reference and final comparisons; principal tools are load balancing, health check design, dependency analysis, cascade prevention.

KPI baseline, target, method and pass contract

ID	MEASURE	BASELINE	DIRECTION	UNIT	TARGET / PASS RULE	CONDITIONS	MEASUREMENT METHOD	GUARDRAIL
KPI-1	check accuracy against request success	O2 reference measured with the same evidence, workload and resource envelope	higher is better	%	$\geq$ O2 reference + 3 percentage points, or within 1 point while improving the primary resource KPI by $\geq$ 20%	AC-1, AC-3	Use versioned fixtures and an independent oracle; report condition-wise values, repeated trials and uncertainty.	No critical negative test, data-integrity failure or unapproved requirement regression is permitted.
KPI-2	cascade failures avoided	O2 reference measured with the same evidence, workload and resource envelope	improve or demonstrate non-inferiority to the O2 reference	count	demonstrate $\geq$ 5% relative improvement, or non-inferiority within the pre-registered tolerance with a $\geq$ 20% gain on the paired resource KPI	AC-1, AC-3	Use the versioned benchmark and independent oracle defined in the project protocol; repeat each stochastic condition at least three times.	No critical negative test, data-integrity failure or unapproved requirement regression is permitted.
KPI-3	unhealthy backend removal latency	O2 reference measured with the same evidence, workload and resource envelope	lower is better	s	$p95 \leq 0.80 \times$ O2 reference at equal or better functional quality	AC-1, AC-4	Warm the system, run at least 30 measured trials per condition with a monotonic clock, and report p50, p95, p99 and failures.	The primary correctness, safety or quality metric may not regress by more than the approved non-inferiority margin.
KPI-4	successful recovery rate under fault injection (%)	O2 reference measured with the same evidence, workload and resource envelope	higher is better	%	$\geq$ O2 reference + 3 percentage points, or within 1 point while improving the primary resource KPI by $\geq$ 20%	AC-1, AC-3	Use versioned fixtures and an independent oracle; report condition-wise values, repeated trials and uncertainty.	No critical negative test, data-integrity failure or unapproved requirement regression is permitted.
KPI-5	service error rate at declared load (%)	O2 reference measured with the same evidence, workload and resource envelope	lower is better	%	$\leq 0.80 \times$ O2 reference; no critical condition may be worse than the reference	AC-1, AC-3	Use versioned fixtures and an independent oracle; report condition-wise values, repeated trials and uncertainty.	No critical negative test, data-integrity failure or unapproved requirement regression is permitted.
KPI-6	request latency p95 (ms)	O2 reference measured with the	lower is better	ms	$p95 \leq 0.80 \times$ O2 reference at equal or better	AC-1, AC-4	Warm the system, run at least 30 measured trials	The primary correctness, safety or quality metric

ID	MEASURE	BASELINE	DIRECTION	UNIT	TARGET / PASS RULE	CONDITIONS	MEASUREMENT METHOD	GUARDRAIL
		same evidence, workload and resource envelope			functional quality		per condition with a monotonic clock, and report p50, p95, p99 and failures.	may not regress by more than the approved non- inferiority margin.

Mandatory negative-test and recovery campaign

NT- 1 The cloud or distributed system encounters listening socket treated as health during the Load Balancer Health Checking That Reflects Real Service Capability operating or validation workflow.

Trigger: Create a controlled fixture, sample, workload or operating condition that reproduces this failure: “The cloud or distributed system encounters listening socket treated as health during the Load Balancer Health Checking That Reflects Real Service Capability operating or validation workflow”.

Expected safe behaviour: The service must remain within the approved degraded-mode contract or fail closed, with no silent loss, duplicate work or unreconciled state.

Recovery: Reconcile desired and observed state, replay safely from the last durable checkpoint and prove that no request or state was lost or duplicated.

Evidence: Retain fixture or sample ID, precondition, version and configuration, expected versus observed result, logs or measurements, recovery time and residual-risk decision.

NT- 2 The cloud or distributed system encounters exhausted connection pools passing checks during the Load Balancer Health Checking That Reflects Real Service Capability operating or validation workflow.

Trigger: Create a controlled fixture, sample, workload or operating condition that reproduces this failure: “The cloud or distributed system encounters exhausted connection pools passing checks during the Load Balancer Health Checking That Reflects Real Service Capability operating or validation workflow”.

Expected safe behaviour: The service must remain within the approved degraded-mode contract or fail closed, with no silent loss, duplicate work or unreconciled state.

Recovery: Reconcile desired and observed state, replay safely from the last durable checkpoint and prove that no request or state was lost or duplicated.

Evidence: Retain fixture or sample ID, precondition, version and configuration, expected versus observed result, logs or measurements, recovery time and residual-risk decision.

NT- 3 The cloud or distributed system encounters deep checks cascading failures during the Load Balancer Health Checking That Reflects Real Service Capability operating or validation workflow.

Trigger: Create a controlled fixture, sample, workload or operating condition that reproduces this failure: “The cloud or distributed system encounters deep checks cascading failures during the Load Balancer Health Checking That Reflects Real Service Capability operating or validation workflow”.

Expected safe behaviour: The service must remain within the approved degraded-mode contract or fail closed, with no silent loss, duplicate work or unreconciled state.

Recovery: Reconcile desired and observed state, replay safely from the last durable checkpoint and prove that no request or state was lost or duplicated.

Evidence: Retain fixture or sample ID, precondition, version and configuration, expected versus observed result, logs or measurements, recovery time and residual-risk decision.

NT- 4 The cloud or distributed system encounters A control-plane or dependency outage prevents bounded recovery in Load Balancer Health Checking That Reflects Real Service Capability during the Load Balancer Health Checking That Reflects Real Service Capability operating or validation workflow.

Trigger: Create a controlled fixture, sample, workload or operating condition that reproduces this failure: “The cloud or distributed system encounters A control-plane or dependency outage prevents bounded recovery in Load Balancer Health Checking That Reflects Real Service Capability during the Load Balancer Health Checking That Reflects Real Service Capability operating or validation workflow”.

Expected safe behaviour: The service must remain within the approved degraded-mode contract or fail closed, with no silent loss, duplicate work or unreconciled state.

Recovery: Reconcile desired and observed state, replay safely from the last durable checkpoint and prove that no request or state was lost or duplicated.

Evidence: Retain fixture or sample ID, precondition, version and configuration, expected versus observed result, logs or measurements, recovery time and residual-risk decision.

NT- 5 Configuration drift or an unsafe rollout bypasses the declared policy in Load Balancer Health Checking That Reflects Real Service Capability.

Trigger: Create a controlled fixture, sample, workload or operating condition that reproduces this failure: “Configuration drift or an unsafe rollout bypasses the declared policy in Load Balancer Health Checking That Reflects Real Service Capability”.

Expected safe behaviour: The service must remain within the approved degraded-mode contract or fail closed, with no silent loss, duplicate work or unreconciled state.

Recovery: Reconcile desired and observed state, replay safely from the last durable checkpoint and prove that no request or state was lost or duplicated.

Evidence: Retain fixture or sample ID, precondition, version and configuration, expected versus observed result, logs or measurements, recovery time and residual-risk decision.

Evidence and reproducibility contract

PRIMARY EVIDENCE

Backends with induced dependency failures and traffic workloads

PROTOCOL AND QUALITY

Version the cited evidence and project inputs, reproduce “Demonstrate a backend passing checks while failing requests”, implement “Design health checks reflecting real service capability”, and compare both under AC-1 to AC-4 without changing the approved evidence or resource envelope. Check workload representativeness, configuration versions, clock alignment, trace completeness, fault labels and recovered state.

PROVENANCE AND INDEPENDENT PARTITION

The Load Balancer Health Checking That Reflects Real Service Capability evidence manifest must record source URL, access date, licence or permission, owner/custodian, record/sample/fixture identifiers, exclusions, transformations, configuration and the objective or KPI using each item. Reserve the AC-3 evidence before O3 tuning and separate it by the independent unit appropriate to cloud and platform engineering; no transformation may leak acceptance information into development.

REPRODUCIBILITY

A clean environment must rebuild the O2 reference, O3 contribution, integrated system and acceptance evidence for Load Balancer Health Checking That Reflects Real Service Capability using the version-pinned toolchain (load balancing, health check design, dependency analysis, cascade prevention), seeds/configuration and an automated or fully scripted runbook.

Mandatory final deliverables

- D1 — Approved Load Balancer Health Checking That Reflects Real Service Capability problem charter, requirements, system boundary, architecture and source-to-claim map.
- D2 — Reproducible Demonstrate a backend passing checks while failing requests with fixtures or protocol, oracle, tests, baseline results and resource profile.
- D3 — Independently testable Health checks reflecting real service capability with design rationale, comparative evidence, trade-offs and limitations.
- D4 — Complete Load Balancer Health Checking That Reflects Real Service Capability prototype or experimental system with configuration, interfaces, telemetry, degraded operation and recovery.
- D5 — Automated or documented acceptance harness covering AC-1 to AC-4, all KPIs and every negative test.
- D6 — Versioned repository or protocol package, evidence manifest, installation/operating guide, API/interface documentation and reproducibility instructions.
- D7 — Final report, research paper or technical article, poster, portfolio case study, demonstration video, video logs and individual contribution/viva evidence.